

NetworkPolicy: Isolamento e Controllo del Traffico

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Spiegare il comportamento di default del networking in OCP (tutto il traffico tra Pod è permesso)
 - Implementare il pattern di sicurezza deny-all + allow selettivo con NetworkPolicy
 - Creare policy ingress che limitano il traffico per **podSelector** e **namespaceSelector**
 - Configurare policy egress per controllare il traffico uscente dai Pod
 - Descrivere **AdminNetworkPolicy** introdotto in OCP 4.14+ per policy a livello cluster
-

Teoria e Concetti Chiave

Comportamento di Default in OCP

In assenza di NetworkPolicy, **tutto il traffico tra Pod è permesso** in OpenShift:

- Pod nello stesso namespace comunicano liberamente
- Pod in namespace diversi comunicano liberamente
- I Pod possono raggiungere internet (se non ci sono firewall perimetrali)

 **Attenzione:** Questo comportamento open-by-default è il contrario del principio di minimo privilegio. Per ambienti di produzione è fondamentale implementare NetworkPolicy.

Cos'è una NetworkPolicy

Una **NetworkPolicy** è una risorsa Kubernetes che definisce regole di **firewall L3/L4** a livello di Pod:

- Seleziona i Pod **target** con **podSelector** (a chi si applicano le regole)
- Definisce regole **ingress** (traffico **entrante** nei Pod target)
- Definisce regole **egress** (traffico **uscente** dai Pod target)
- Le regole usano **podSelector**, **namespaceSelector** e **ipBlock** come sorgente/destinazione
- Supporta filtraggio per **porta e protocollo** (TCP/UDP)

Importante: logica OR tra NetworkPolicy

Se più NetworkPolicy selezionano lo stesso Pod, le loro regole vengono combinate con logica **OR** (unione): se almeno una policy permette il traffico, il traffico è permesso.

Pattern Fondamentale: Deny-All + Allow Selettivo

Il pattern di sicurezza consigliato è:

1. **Applicare una policy deny-all** per ingress (e opzionalmente egress) a tutti i Pod del namespace
2. **Aggiungere policy allow specifiche** solo per il traffico necessario

```
Step 1: Deny-all ingress → tutti i Pod non accettano traffico dall'esterno  
Step 2: Allow da frontend → solo il frontend può accedere al backend  
Step 3: Allow da monitoring → Prometheus può fare scraping su /metrics
```

Selezione dei Pod: podSelector e namespaceSelector

podSelector: seleziona Pod per label, nello stesso namespace della policy

```
podSelector:  
  matchLabels:  
    app: backend    # ← si applica ai Pod con label app=backend
```

namespaceSelector: seleziona Pod in namespace con specifici label

```
namespaceSelector:  
  matchLabels:  
    environment: production    # ← accetta traffico da namespace con questo label
```

Combinazione (AND): entrambi devono corrispondere

```
- namespaceSelector:  
  matchLabels:  
    kubernetes.io/metadata.name: frontend-ns  
podSelector:  
  matchLabels:  
    app: frontend    # ← solo Pod "frontend" nel namespace "frontend-ns"
```

Combinazione (OR): due elementi separati nella lista **from/to**

```
from:
- namespaceSelector:           # ← OR
  matchLabels:
    environment: production
- podSelector:                 # ← OR
  matchLabels:
    app: monitoring
```

ipBlock: Policy per CIDR

Per consentire traffico da IP esterni (es. rete aziendale) o verso internet:

```
ingress:
- from:
  - ipBlock:
      cidr: 10.0.0.0/8           # ← permette dalla rete aziendale
      except:
        - 10.0.1.0/24          # ← eccetto questa subnet
```

AdminNetworkPolicy (OCP 4.14+)

AdminNetworkPolicy (ANP) è una CRD introdotta in OCP 4.14+ che permette agli amministratori del cluster di definire policy **a livello cluster** (non namespace). Ha priorità più alta rispetto alle NetworkPolicy degli utenti:

```
Ordine di valutazione: AdminNetworkPolicy → BaselineAdminNetworkPolicy →
NetworkPolicy → Default allow
```

Casi d'uso ANP:

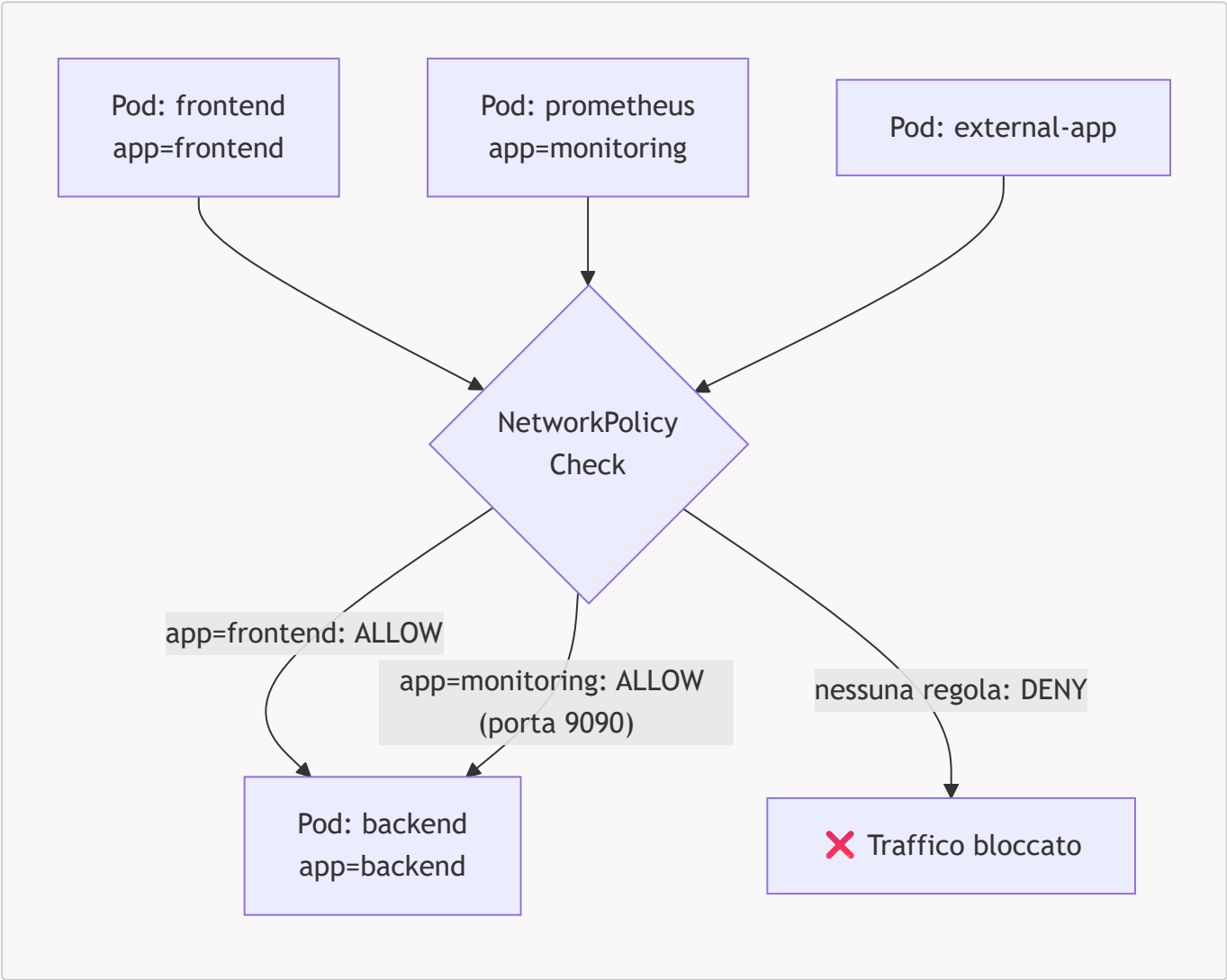
- Bloccare il traffico tra namespace di ambienti diversi (prod/dev)
 - Permettere sempre il traffico di monitoring cluster-wide
 - Definire regole di sicurezza che gli utenti non possono sovrascrivere
-

Tabella: Confronto NetworkPolicy, AdminNetworkPolicy, BaselineAdminNetworkPolicy

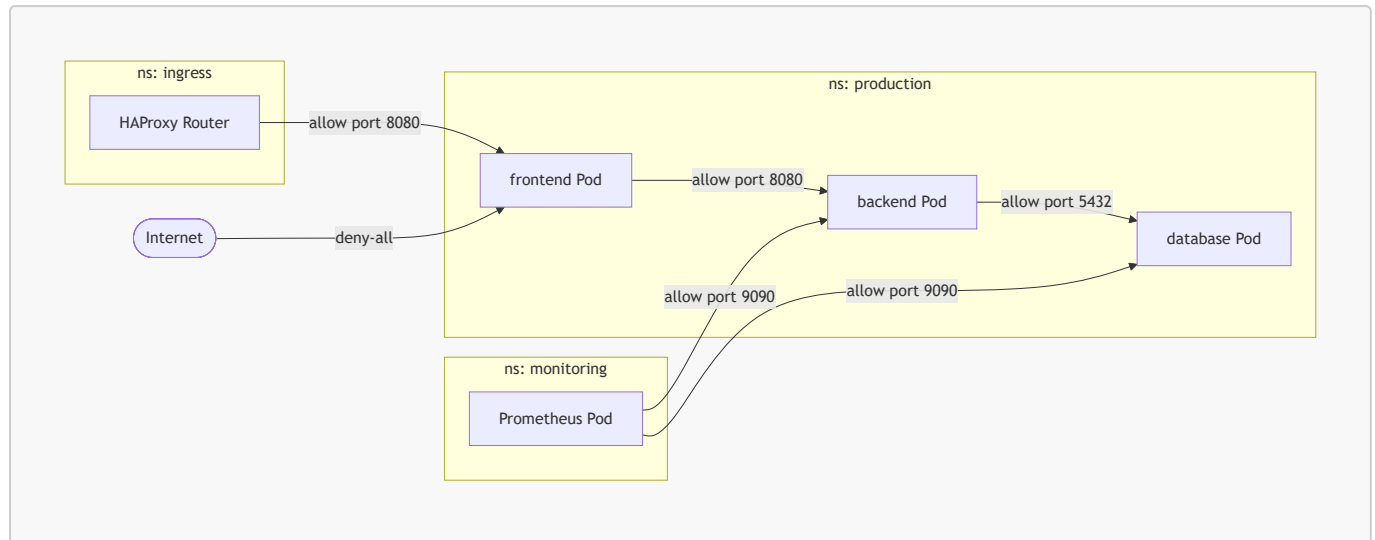
Aspetto	NetworkPolicy	AdminNetworkPolicy	BaselineAdminNetworkPolicy
Gestito da	Utente (namespace admin)	Cluster admin	Cluster admin
Scope	Namespace	Cluster-wide	Cluster-wide (default fallback)
Priorità	Media	Alta (override NP)	Bassa (fallback se no NP)
Disponibilità	OCP 4.x	OCP 4.14+ (OVN required)	OCP 4.14+
Azione	Allow/Deny	Allow/Deny/Pass	Allow/Deny

Diagrammi

Flusso con NetworkPolicy Deny-All + Allow Selettivo



Topologia NetworkPolicy in un Ambiente Multi-Namespace



Comandi Pratici con Output Atteso

```
# Visualizzare tutte le NetworkPolicy nel namespace
oc get networkpolicy -n myproject
```

```
# Output atteso:
# NAME                POD-SELECTOR  AGE
# deny-all-ingress   <none>       5m
# allow-from-frontend app=backend   3m
# allow-monitoring    app=backend   2m
```

```
# Dettaglio di una NetworkPolicy
oc describe networkpolicy allow-from-frontend -n myproject
```

```
# Output atteso:
# Name:                allow-from-frontend
# Namespace:           myproject
# PodSelector:         app=backend
# Allowing ingress traffic:
#   To Port: 8080/TCP
#   From:
#     PodSelector: app=frontend
```

```
# Applicare una NetworkPolicy da file YAML
oc apply -f deny-all-ingress.yaml
```

```
# Output atteso:
# networkpolicy.networking.k8s.io/deny-all-ingress created
```

```
# Testare la connettività da un Pod a un altro (dovrebbe fallire con deny-all)
oc exec -it frontend-pod -n myproject -- curl --max-time 5
http://backend:8080/healthz
```

```
# Output atteso (prima della policy allow):
# curl: (28) Connection timed out after 5001 milliseconds
```

```
# Testare la connettività dopo aver applicato la policy allow
oc exec -it frontend-pod -n myproject -- curl --max-time 5
http://backend:8080/healthz
```

```
# Output atteso (dopo la policy allow):
# {"status":"ok"}
```

```
# Verificare la label del namespace (necessaria per namespaceSelector)
oc get namespace myproject --show-labels
```

```
# Output atteso:
# NAME          STATUS    AGE    LABELS
# myproject     Active    10d
kubernetes.io/metadata.name=myproject,environment=production
```

```
# Aggiungere una label al namespace per usarla in namespaceSelector
oc label namespace monitoring-ns purpose=monitoring
```

```
# Output atteso:
# namespace/monitoring-ns labeled
```

```
# Elencare le AdminNetworkPolicy (OCP 4.14+ con OVN-Kubernetes)
oc get adminnetworkpolicy
```

```
# Output atteso:
# NAME                                PRIORITY    AGE
# deny-cross-env-traffic              10          2d
```

Esempi YAML Completi

Deny-All Ingress (Pattern Fondamentale)

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-ingress
  namespace: myproject      # ← si applica a tutti i Pod in questo namespace
spec:
  podSelector: {}           # ← {} seleziona TUTTI i Pod nel namespace
  policyTypes:
    - Ingress               # ← questa policy riguarda il traffico entrante
  # ingress: [] assente → nessun traffico ingress permesso (deny-all)

```

Allow Ingress dal Frontend e dal Monitoring

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-backend-ingress
  namespace: myproject
spec:
  podSelector:
    matchLabels:
      app: backend           # ← si applica ai Pod con label app=backend
  policyTypes:
    - Ingress
  ingress:
    # Regola 1: permette traffico dai Pod frontend sullo stesso namespace
    - from:
      - podSelector:
          matchLabels:
            app: frontend     # ← solo Pod con label app=frontend
      ports:
      - protocol: TCP
        port: 8080           # ← solo sulla porta 8080
    # Regola 2: permette traffico da Prometheus nel namespace monitoring
    - from:
      - namespaceSelector:    # ← AND: namespace + pod devono corrispondere
          matchLabels:
            purpose: monitoring
        podSelector:
          matchLabels:
            app: prometheus
      ports:
      - protocol: TCP
        port: 9090           # ← porta metrics di Prometheus scraping

```

Deny-All Egress + Allow Selettivo in Uscita

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-egress-policy
  namespace: myproject
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Egress # ← controlla il traffico uscente
  egress:
    # Permette al backend di accedere al database (stesso namespace)
    - to:
        - podSelector:
            matchLabels:
              app: database
        ports:
          - protocol: TCP
            port: 5432 # ← PostgreSQL
    # Permette la risoluzione DNS (CoreDNS nel namespace openshift-dns)
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: openshift-dns
        ports:
          - protocol: UDP
            port: 53 # ← DNS (sempre necessario per risolvere nomi
Service)
          - protocol: TCP
            port: 53
    # Permette HTTPS verso internet per download aggiornamenti
    - to:
        - ipBlock:
            cidr: 0.0.0.0/0 # ← qualsiasi IP esterno
            except:
              - 10.0.0.0/8 # ← eccetto la rete privata (comunicazione interna
tramite Service)
              - 172.16.0.0/12
              - 192.168.0.0/16
        ports:
          - protocol: TCP
            port: 443 # ← solo HTTPS

```

AdminNetworkPolicy (OCP 4.14+)

```

apiVersion: policy.networking.k8s.io/v1alpha1
kind: AdminNetworkPolicy

```



```
metadata:
  name: deny-cross-environment
spec:
  priority: 10 # ← priorità: più basso = valutato prima (0-1000)
  subject:
    namespaces:
      matchLabels:
        environment: production # ← si applica ai namespace di produzione
  ingress:
    # Nega traffico da namespace di sviluppo verso produzione
    - name: "deny-from-dev"
      action: Deny # ← Deny, Allow, o Pass
      from:
        - namespaces:
            matchLabels:
              environment: development
    # Permette traffico dal namespace di monitoring (override delle NP degli utenti)
    - name: "allow-monitoring"
      action: Allow
      from:
        - namespaces:
            matchLabels:
              purpose: monitoring
  ports:
    - portNumber:
        protocol: TCP
        port: 9090
```

Note Specifiche per Azure ARO

OVN-Kubernetes e NetworkPolicy su ARO

ARO usa OVN-Kubernetes, che implementa NetworkPolicy tramite **ACL (Access Control Lists) OVN** invece di iptables. Questo offre:

- **Pieno supporto NetworkPolicy** (incluso egress, a differenza di OpenShift SDN)
- **AdminNetworkPolicy** supportata nativamente
- Performance migliore rispetto a iptables per cluster con molte policy

```
# Verificare che OVN stia applicando le ACL
oc debug node/worker-0 -- chroot /host \
  ovn-nbctl list ACL | grep -A 5 "name.*deny-all-ingress"
```

Namespace di Sistema su ARO

Su ARO, i namespace di sistema (es. `openshift-monitoring`, `openshift-ingress`) hanno label predefinite che permettono di costruire NetworkPolicy che li includono:

```
# Visualizzare le label dei namespace di sistema
oc get namespace openshift-monitoring --show-labels
# Labels: kubernetes.io/metadata.name=openshift-monitoring,
#        openshift.io/cluster-monitoring=true

# Usare queste label nelle NetworkPolicy
# namespaceSelector:
#   matchLabels:
#     kubernetes.io/metadata.name: openshift-monitoring
```

 **Suggerimento:** Prima di applicare una policy deny-all in produzione su ARO, verificare che le policy allow per `openshift-monitoring` e `openshift-ingress` siano in place, altrimenti il monitoring e il routing si interrompono.

Debugging NetworkPolicy su ARO

```
# Verificare se una policy sta bloccando del traffico
# Usare un Pod temporaneo per testare la connettività
oc run test-connectivity --image=registry.access.redhat.com/ubi9/ubi-minimal:9.4 \
  --rm -it --restart=Never -- curl --max-time 5 http://backend:8080/healthz

# Se il test fallisce, ispezionare le NetworkPolicy che selezionano il Pod
# destinazione
oc get networkpolicy -n myproject -o yaml | grep -A 10 "podSelector"
```

Riepilogo dei Punti Chiave

- In assenza di NetworkPolicy, tutto il traffico tra Pod è permesso in OpenShift (default open)
 - Il pattern consigliato è **deny-all ingress + allow selettivo**: prima si blocca tutto, poi si apre solo ciò che serve
 - Le regole **from/to** usano **podSelector** (stesso namespace), **namespaceSelector** (namespace per label) e **ipBlock** (CIDR)
 - Più NetworkPolicy sullo stesso Pod vengono combinate con logica OR (se una permette, il traffico passa)
 - La policy egress controlla il traffico uscente: ricordare sempre di permettere UDP/TCP porta 53 per il DNS
 - **AdminNetworkPolicy** (OCP 4.14+) permette policy cluster-wide con priorità superiore alle NetworkPolicy degli utenti
 - Su ARO con OVN-Kubernetes, le NetworkPolicy sono implementate tramite ACL OVN (supporto completo egress incluso)
-

Domande di Verifica

1. Un Pod `backend` con `deny-all-ingress` NetworkPolicy applicata non riceve traffico dal Pod `frontend`. Cosa occorre fare?

- A) Rimuovere la NetworkPolicy `deny-all` e usarne una permissiva
- B) Aggiungere una NetworkPolicy separata che permette traffico da `app=frontend` al Pod `app=backend` ✓
- C) Cambiare il tipo di Service da ClusterIP a NodePort
- D) Aggiungere un'annotazione al Pod `frontend`

2. Una NetworkPolicy egress su `app=backend` blocca la risoluzione DNS. Quale regola mancante causa il problema?

- A) Una regola egress allow per la porta TCP 80 verso `openshift-dns`
- B) Una regola egress allow per le porte UDP/TCP 53 verso il namespace `openshift-dns` ✓
- C) Una regola ingress allow dalla porta 53
- D) Un'annotazione `dns.openshift.io/policy: allow`

3. Si vogliono isolare due namespace `production` e `staging` per impedire la comunicazione tra loro. Quale approccio è più scalabile?

- A) NetworkPolicy per ogni coppia di namespace
- B) Applicare `deny-all` in ogni namespace e non aggiungere regole tra essi ✓
- C) Usare NSG Azure per bloccare il traffico tra i namespace
- D) Separare i namespace in VNet Azure diverse

4. Su ARO con OVN-Kubernetes, quale vantaggio ha l'implementazione delle NetworkPolicy rispetto a OpenShift SDN?

- A) Supporto per Route TLS passthrough
- B) Performance migliore e supporto completo per egress NetworkPolicy ✓
- C) Possibilità di usare annotazioni `nginx` sulle Route
- D) Integrazione con Azure NSG

5. Quale CRD introdotta in OCP 4.14+ permette a un cluster admin di definire policy di rete con priorità superiore alle NetworkPolicy degli utenti?

- A) `ClusterNetworkPolicy`
- B) `GlobalNetworkPolicy`
- C) `AdminNetworkPolicy` ✓
- D) `IngressNetworkPolicy`